WEBINAR · WORKSHOP SERIES

Embedded Programming for Robotics

Md Tasbeer Ahmed
Lab Instructor
Dept. of ECE
North South University

AGENDA

What this hour looks like

1. **What embedded programming actually is**

   The idea behind every robot

2. **The boards you can start with**

   Arduino, Pico, ESP32 — and which to buy

3. **The software: IDEs**

   Four options, live tour of each

4. **Connecting the board to your computer**

   The step everyone gets stuck on

5. **Writing your first programs**

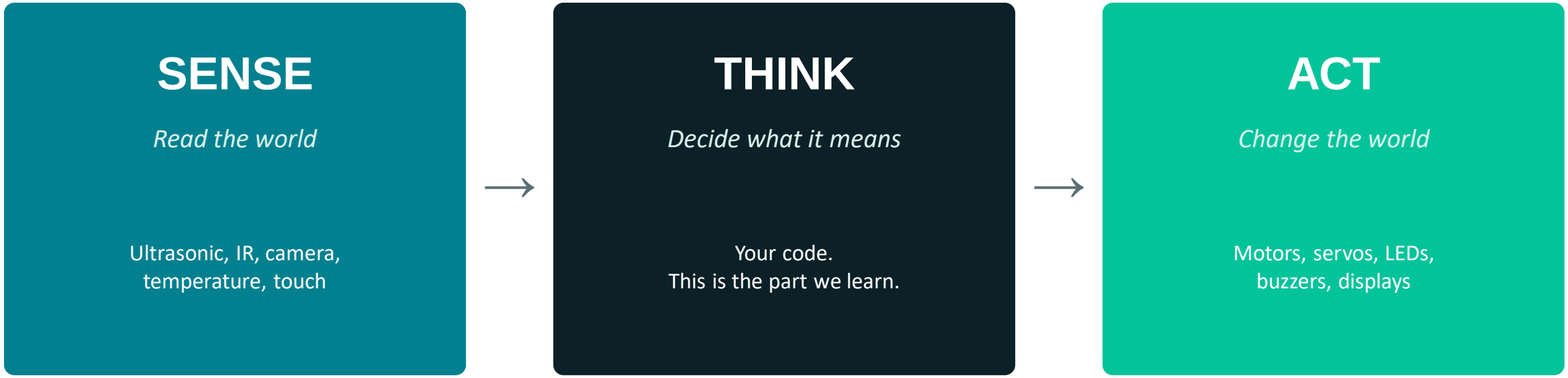
   Blink → read a sensor → make a decision

6. **Real projects**

   What all this adds up to

THE CORE IDEA

Every robot does the same three things



Embedded programming is how you design the middle box, the part that turns a number from a sensor into a command for a motor.

THE HARDWARE

So what is a microcontroller?

A tiny computer on a single chip.

It has no screen, no keyboard, and no operating system. It does exactly one thing: run the program you gave it, forever, until the power goes off.

It costs a few dollars, boots in milliseconds, and is inside your microwave, your car, your washing machine, and every robot you have ever seen.

Laptop

Runs many programs, has an OS, boots in ~30 s

Microcontroller

Runs ONE program, no OS, boots instantly

Because it runs only your program, nothing can slow it down or interrupt it. That is why robots use them.

THE HARDWARE

Three boards worth knowing

Arduino Uno

The classic starter

- Easiest to learn
- Huge tutorial base
- Very forgiving of mistakes
- No WiFi, modest speed

Raspberry Pi Pico / Pico W

The modern value pick

- Very cheap, much faster
- Runs MicroPython or C++
- Pico W adds WiFi
- Slightly newer, fewer tutorials

ESP32

The connected one

- WiFi + Bluetooth built in
- Powerful, many pins
- Best for IoT projects
- Steeper learning curve

Starting today with no experience? Arduino Uno. Already comfortable and want more power for the same money? Pico.

THE SOFTWARE

What is an IDE?

Integrated Development Environment

The one program you install on your computer to write code and send it to your board. It bundles four jobs together:

1

Editor

Where you type your program

2

Compiler

Translates your text into machine instructions

3

Uploader

Sends those instructions to the board over USB

4

Monitor

Shows you messages the board sends back

You do not need to understand the compiler. You need to know which button does what and that is what we'll look at next.

Arduino IDE

The default starting point. Free, works on Windows / macOS / Linux, supports Arduino, ESP32 and Pico.

`Verify (✓)`	Checks your code for errors. Does not touch the board.
`Upload (→)`	Compiles, then sends the program to the board.
`Serial Monitor`	Opens a window where the board can print messages to you.
`Tools ▸ Board`	Tell the IDE which board you are using.
`Tools ▸ Port`	Tell the IDE which USB socket the board is plugged into.

BEST FOR

- Absolute beginners
- Arduino boards
- Following tutorials
- Quick one-file sketches

Language: C++
Download: arduino.cc/en/software

Thonny: for MicroPython on the Pico

A much simpler editor built for Python beginners. If you use a Raspberry Pi Pico, this is the gentlest possible path.

Why beginners like it

- Python is easier to read than C++
- No 'compile' step needed, code runs instantly
- A live shell: type one line, see it happen
- Detects the Pico automatically
- Error messages in plain English

Same blink, in MicroPython:

```
from machine import Pin
import time

led = Pin(25, Pin.OUT)

while True:
    led.value(1)
    time.sleep(1)
    led.value(0)
    time.sleep(1)
```

VS Code + PlatformIO

The professional setup. More to learn up front, far better once your projects grow past one file.

Worth it for

- Autocomplete that actually works
- Jump to any function definition
- Proper multi-file projects
- Git version control built in
- One tool for every board

The trade-off

- Heavier install
- More settings to understand
- Overkill for a first blink sketch

Suggestion: start on the Arduino IDE. Move here when you find yourself annoyed by the Arduino IDE — not before.

This is what I use for lab and research work.

IDE TOUR · 4 OF 4

TinkerCAD or Wokwi

A full online simulator that runs in your browser. Nothing to install. No hardware to buy. You can start tonight.

Free

No account needed to start

Instant

Runs in a browser tab

Realistic

Simulates real sensors and displays

Shareable

Send a link, anyone can run your circuit

If you take one thing from this section: go to tinkercad.com or wokwi.com after this webinar and run the blink example. You will have written and run your first embedded program within five minutes, with zero setup.

CHOOSING

Which one should you use?

	Arduino IDE	Thonny	VS Code + PIO	TinkerCAD/Wokwi
Difficulty	Easy	Easiest	Harder	Easiest
Language	C++	MicroPython	C++ / Python	C++ / Python
Install needed	Yes	Yes	Yes	No
Hardware needed	Yes	Yes	Yes	No
Best board	Arduino, ESP32	Pico	Any	Any
Good for	Learning	Python learners	Real projects	Trying it out

There is no wrong answer. Every one of these will teach you the same ideas.

THE PART EVERYONE GETS STUCK ON

Connecting your board to your computer

1

Install the driver

Most boards work immediately. Cheaper clones need a USB-to-serial driver: CH340 or CP2102. Install it once, forget about it forever.

2

Plug in with a DATA cable

Many USB cables only carry power. If nothing appears, try a different cable before anything else.

3

Select your board

Tools ▸ Board. Tell the IDE exactly which board you have, so it compiles the right machine code.

4

Select the port

Tools ▸ Port. Windows shows COM3, COM4… macOS and Linux show /dev/tty… Unplug the board and see which one disappears — that's yours.

Board selected + port selected = you are ready to upload. That is the whole setup.

TROUBLESHOOTING

When the upload fails

No port showing up

Bad cable (power-only), or driver not installed. Swap the cable first.

Port greyed out

Board not detected — same two causes as above.

'Access denied' / port busy

The Serial Monitor is already using it. Close it and upload again.

Uploads but nothing happens

Wrong board selected, or your wiring doesn't match your pin numbers.

Garbled text in the monitor

Baud rate mismatch. The monitor must match Serial.begin(9600).

Board resets when the motor starts

Power problem, not code. Motors need their own supply.

Almost every failure on this list is a cable, a driver, or a wrong menu selection — not your code.

PART TWO

Now let's actually write something

Four short programs. Each one adds exactly one new idea.

Every program has the same two parts

`setup()`
Runs ONCE when the board powers on. "Get ready."

`loop()`
Runs FOREVER, over and over. "Do the job."

Putting on your shoes happens once.
Walking is one step, repeated forever.

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);   // LED on
  delay(1000);              // wait 1 s
  digitalWrite(13, LOW);    // LED off
  delay(1000);              // wait 1 s
}
```

That is a complete, working program. Nine lines.

The five things in that program

`pinMode(13, OUTPUT)`	"Pin 13, you will send signals out, not receive them."
`digitalWrite(13, HIGH)`	HIGH means on, LOW means off. That is all they mean.
`delay(1000)`	Wait. The number is in milliseconds, so 1000 = 1 second.
`; (semicolon)`	Ends every instruction. Like a full stop in a sentence.
`// two slashes`	A note for humans. The board ignores everything after them.

Change delay(1000) to delay(100) and the LED blinks ten times faster. That is the moment you realise you are in control.

Making the system talk to you

A variable is a labelled box that holds a number.

```
int speed = 100;
```

int — the box holds a whole number
speed — the name you chose for it
= 100 — what you put inside it

Serial.println() prints a number to your screen.

This is your entire debugging toolkit as a beginner. When the robot misbehaves, print the numbers and look at them.

```
void setup() {
  Serial.begin(9600);   // open the channel
}

int count = 0;

void loop() {
  count = count + 1;
  Serial.print("Count: ");
  Serial.println(count);
  delay(500);
}
```

```
Output:  Count: 1   Count: 2   Count: 3 …
```

Taking a real reading

An ultrasonic sensor sends out a sound pulse and measures how long the echo takes to come back. Sound travels at a known speed — so time tells us distance. Exactly how a bat sees.

```
int getDistance() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long duration = pulseIn(ECHO, HIGH);
  return duration / 58;
}

void loop() {
  int distance = getDistance();

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
  delay(200);
}
```

The one line that matters

```
int distance = getDistance();
```

The sensor goes and measures. The equals sign puts that measurement into our box called distance.

The number is now yours. You can print it, compare it, and act on it. That is the whole bridge between the physical world and your program.

PROGRAM 4 · IF / ELSE

Making a decision

Say the rule in English first: "If the distance is less than 20 centimetres, turn on the LED."

```
if (distance < 20) {
  digitalWrite(LED, HIGH);
  Serial.println("Object detected!");
} else {
  digitalWrite(LED, LOW);
}
```

The code is almost the same sentence.

Three levels instead of two — just add else if:

```
if (distance < 10)      Serial.println("DANGER");
else if (distance < 30) Serial.println("Careful");
else                    Serial.println("All clear");
```

The comparisons you need

`<`	less than
`>`	greater than
`==`	equal to (two signs!)
`!=`	not equal to

Watch out: = puts a value in a box.
== asks a question. Everyone mixes these up.

IT ALL COMES TOGETHER

A complete robot brain

```
void loop() {
  int distance = getDistance();      // SENSE
  Serial.println(distance);

  if (distance < 20) {               // THINK
    stopRobot();                     // ACT
    digitalWrite(LED, HIGH);
  } else {
    forward();
    digitalWrite(LED, LOW);
  }

  delay(100);
}
```

SENSE

Read the sensor into a variable

THINK

Compare it with if / else

ACT

Tell the motors what to do

Same three boxes from the first five minutes — and there is nothing in this program you haven't already seen.

PART THREE

Where this goes

Real projects built with exactly the ideas you just saw — a sensor, a variable, and a decision.

PROJECT · 1 OF 4

Ball-and-beam balancing system

What it does

A ball rolls freely on a tilting beam. The robot keeps it balanced at a target position i.e. continuously, without ever letting it fall off.

Built from

- Arduino + HC-SR04 ultrasonic sensor
- MG996R servo to tilt the beam
- PID control, tuned live from MATLAB
- A GUI with sliders over the serial link

The idea you already know

The very same loop: read the sensor, compare it to where you want the ball to be, move the servo. The only new part is deciding how far to move — a little for a small error, more for a large one.

That idea is called feedback control, and it is the single most useful concept in robotics.

PROJECT · 2 OF 4

Human-interactive waste-bin robots

What it does

A fleet of four small robots that notice people, react, and encourage them to throw waste in the bin — a study in how robots can change human behaviour.

Built from

- Raspberry Pi Pico per robot
- PCA9685 driver for many servos
- DFPlayer Mini for sound
- MicroPython firmware

The idea you already know

Each robot is running an if / else on a sensor reading — is someone nearby? — and choosing a movement and a sound in response.

What makes it research rather than a demo is the question behind it: does a robot that reacts to you actually make you more likely to use the bin?

PROJECT · 3 OF 4

Sensor to dashboard: a people counter

What it does

Counts people passing through a doorway and logs the numbers to a web dashboard you can open from your phone on the same network. No cloud service involved.

Built from

- Raspberry Pi Pico W (WiFi)
- VL53L0X time-of-flight distance sensor
- SSD1306 OLED for a local display
- A tiny web server on the board itself

The idea you already know

Read a distance. If it drops below a threshold, someone walked past so add one to a counter variable. That is Program 2 and Program 4 combined.

Everything after that — the display, the WiFi, the dashboard — is presentation. The intelligence is one comparison and one counter.

PROJECT · 4 OF 4

And a few smaller ones

1

Animated robot eyes

A Pico driving an OLED screen through ten expressions, with natural blinking between them. Pure output — and it gives a robot personality instantly.

2

Smart door monitor

A Pico W and a distance sensor that tells you on your phone whether a door was left open. Two sensors and one if statement.

3

P2P Communication

Using ESP’s Peer to Peer Communication Protocol to send a command from one ESP Board to another ESP Board.

Every one of these started as somebody blinking an LED and wondering what else the board could do.

Thank you

Questions?

Your challenge, if you want one:

Open TinkerCAD and make a Level Detector using an Ultrasonic Sensor and an Arduino (Possibly some LEDs as well).